

Week 4 — Lab Handout (Student)

Validating & Defending Your Routes

Time: ~90 min in-class **Work as:** Your team (5 members) **AI use this week:** X OFF. Still the Fundamentals Wall — write it yourself. **What you leave with:** Every create/update route rejecting bad input with clear, standardized errors.

Why this lab matters

Right now your routes accept anything. Today you make them defensive: bad input is caught and rejected *before* it reaches your logic, with a clear error the interface can display later. This is the difference between an app that crashes and one that fails gracefully — and it's a skill the Week 5 checkpoint may test directly.

Task 1 — Build your validation matrix (20 min)

For every **create** and **update** route, list each field and its rules. Use the vocabulary from lecture: presence, type, length/range, format, allowed values, referential.

Route	Field	Rules
POST /orders	item	required, string, 1-100 chars
POST /orders	qty	required, number, 1-999
POST /orders	status	required, one of

- ☐ Every create/update route has a complete field-by-field rule list.
- ☐ Save it to `/docs/validation.md`.

Task 2 — Implement validation as guard clauses (30 min)

At the **top** of each create/update handler, check the bad cases first and exit early. Pseudocode:

```
function createOrder(request):
  data = request.body
  if data.item is missing:      return response(422, "item is required")
  if length(data.item) > 100:   return response(422, "item too long")
  if data.qty is not a number:  return response(422, "qty must be a number")
  if data.qty < 1 or data.qty > 999: return response(422, "qty out of range")
  if data.status not in {pending, paid, shipped}:
    return response(422, "invalid status")

  # only valid data reaches here
  ... create the order ...
  return response(201, createdOrder)
```

- ☐ Validation sits at the top of each handler (guard-clause style), not scattered.
- ☐ Invalid input returns **422** (bad data), never a 500 crash.
- ☐ Valid input passes straight through.

Task 3 — Standardize your error shape (10 min)

Agree on **one** error response format for the whole team and use it everywhere. Example:

```
{ "status": 422, "error": "qty out of range", "field": "qty" }
```

- ☐ Every validation failure returns the same shape.
- ☐ Messages are human-readable and name the problem field.
- ☐ No raw stack traces or database errors leak to the client.

Task 4 — Add an authorization guard where it matters (10 min)

Pick at least one sensitive action (e.g., delete, or editing someone else's record) and add a simple permission check. Concept-level is

fine — stack-neutral pseudocode:

```
if not currentUserOwns(order): return response(403, "not allowed")
```

- ☐ At least one route has an authorization guard.
- ☐ Forbidden actions return **403**, distinct from validation's 422.

Task 5 — Try to break your own app (10 min)

Send deliberately bad requests and confirm each is handled, not crashed:

- ☐ Missing required field → 422
- ☐ Wrong type (e.g., `qty: "cake"`) → 422
- ☐ Out-of-range value → 422
- ☐ Forbidden action → 403
- ☐ Log each attempt + result in `/docs/validation.md`.

If any bad input returns a 500 or crashes the server, that's a hole — fix it before you leave.

End-of-Lab Checklist

- ☐ `/docs/validation.md` — full matrix + your break-it test log
- ☐ Guard-clause validation on every create/update route, returning 422 on bad data
- ☐ One consistent error shape across the team; no leaked internals
- ☐ At least one authorization guard returning 403
- ☐ Every route owned on the board; all merges via reviewed PR; no AI used

Looking ahead to Week 5

You'll complete the **backend controllers** — the logic that processes validated data and returns real responses — and write **automated tests** for them. That work closes out **Deliverable 2 (Routing, Logic & Tests, 25%)**, and includes the **live AI-off checkpoint** where each of you builds or fixes a route solo. This week's validation reflexes are exactly what that checkpoint looks for.